Numerical Modeling of the Ocean and Atmosphere I (2026)

L. Fernando Pareja-Roman

From Erin to your own WRF application

I wanted to simulate Hurricane Erin, so I first had to describe a region of the atmosphere and supply its initial state and evolving lateral boundaries. WRF cannot begin with a map alone: it needs terrain, land properties, three-dimensional weather fields, and a consistent clock. This exercise follows that preparation, using Erin as a template for a domain you choose.

You will use an already compiled WRF and WPS installation. Your own folder will contain the scripts, links to the programs, namelists, downloaded analyses, and initialized model files. The aim is to explain your choices and inspect the resulting atmosphere before committing to a forecast.

The route from global data to a regional atmosphere

Stage	What enters → what comes out
geogrid.exe	Domain settings + WPS_GEOG static data → geo_em.d01.nc
ungrib.exe	FNL GRIB2 files + Vtable → GFS:YYYY-MM-DD_HH intermediate files
metgrid.exe	geo_em + intermediate weather → met_em.d01.*.nc
real.exe	met_em + namelist.input → wrfinput_d01 and wrfbdy_d01
wrf.exe	Initial state + boundary updates + physics → wrfout forecast files

The exercise submission stops after initialization and verification. A successful real.exe run is not a completed hurricane forecast. We inspect its output before running wrf.exe.

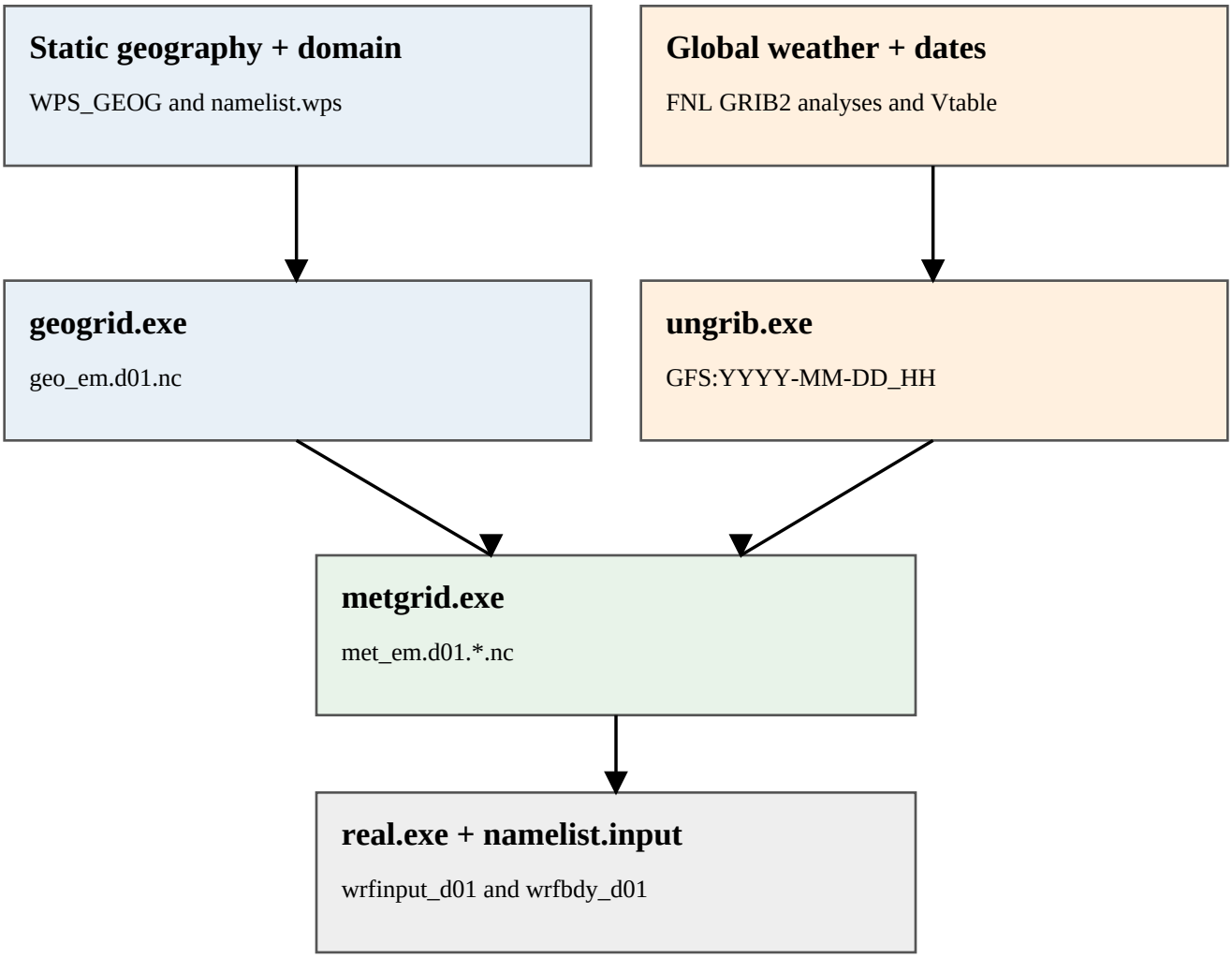
Questions we will explore together

Where your domain lies; how many grid intervals it contains; whether projection distortion is acceptable; which global analyses supply the weather; how WPS and WRF share dates and dimensions; and whether the initial fields look physically reasonable.

Blue introduces decisions and explanations. Gray boxes contain commands or settings. **Orange** identifies a choice that deserves extra attention.

A visual guide to the preparation

Two streams come together in metgrid: the landscape we selected and the global weather for our dates. real.exe then places that weather on WRF's vertical grid.



Following one field through the workflow

An FNL temperature field starts on the global input grid. Ungrib decodes it, metgrid interpolates it horizontally, and real.exe prepares the model state on WRF's vertical coordinate. The verification figures let us inspect that initialized atmosphere before forecasting.

Following one design choice

Changing e_we changes the domain width. Changing dx changes both nominal resolution and width unless the dimensions are adjusted too. The MATLAB map helps us see these consequences before downloading and interpolating the weather.

1. A folder for your application

We can begin by moving into the location where you would like to store your application. A location with room for the analyses and model output works well. From there, the following commands create our working folder:

```
mkdir exercise
cd exercise
module purge
module load nco
module load roms
```

Next, we can make our own copies of the two main scripts and the submission and verification helpers. These are your copies: editing them will not change the shared class template.

```
cp /projects/f_omg_1/lfp33/nmoa_2026/wrf_exercise/build_wrf.sh .
cp /projects/f_omg_1/lfp33/nmoa_2026/wrf_exercise/get_wrf_fnl.sh .
cp /projects/f_omg_1/lfp33/nmoa_2026/wrf_exercise/submit.sh .
cp /projects/f_omg_1/lfp33/nmoa_2026/wrf_exercise/verify_wrf_inputs.py .
cp /projects/f_omg_1/lfp33/nmoa_2026/wrf_exercise/get_wps_domain.m .
```

We will use get_wps_domain.m in MATLAB for domain design. A Python alternative, get_wps_domain.py, is available in the same shared folder.

```
./build_wrf.sh -noclean
ls -l wrf.exe real.exe
```

We will use -noclean here. In this class edition, build_wrf.sh only makes symbolic links to the existing programs and runtime tables. It does not compile WRF. Without -noclean it prints a warning and asks whether to proceed with link creation; the default is no.

Where the shared resources live

The common prefix below is /projects/f_omg_1/lfp33/nmoa_2026.

Resource	Location
Scripts to copy	wrf_exercise/
Compiled WRF	exercise_01/WRF/main/; runtime tables in WRF/run/
Compiled WPS	exercise_01/WPS/geogrid/, ungrib/, metgrid/
Static geography	/home/lfp33/repository/WPS_GEOG
Plotting Python	/home/lfp33/miniconda3/envs/mercator2/bin/python

The shared WPS_GEOG data are large: keep their path rather than copying them. If a shared path gives Permission denied, resolve account access before submission; creating links does not grant permission to the target.

What -noclean adds to our folder

Our class already has a compiled WRF. This version of build_wrf.sh connects our application folder to that installation. The before-and-after examples below show a representative listing; the complete set contains additional tables and links.

Before: our copied scripts

```
exercise/
  build_wrf.sh
  get_wrf_fnl.sh
  get_wps_domain.m
  submit.sh
  verify_wrf_inputs.py
```

At this stage, the folder contains our instructions for preparing the case. The forecast program and its runtime data have not yet been linked here.

```
./build_wrf.sh -noclean
```

After: programs and supporting data are available here

```
exercise/
  build_wrf.sh       get_wrf_fnl.sh       submit.sh
  get_wps_domain.m   verify_wrf_inputs.py
  wrf.exe      -> shared WRF/main/wrf.exe
  real.exe     -> shared WRF/main/real.exe
  LANDUSE.TBL  -> shared WRF/run/LANDUSE.TBL
  VEGPARM.TBL  -> shared WRF/run/VEGPARM.TBL
  SOILPARM.TBL -> shared WRF/run/SOILPARM.TBL
  RRTMG_SW_DATA -> shared WRF/run/RRTMG_SW_DATA
  RRTMG_LW_DATA -> shared WRF/run/RRTMG_LW_DATA
  ... additional runtime links ...
```

These links make the compiled model and its supporting data available in the folder from which we run WRF. wrf.exe advances the atmosphere; the tables and radiation data supply information used by the selected physics. A link works only while its shared target remains accessible.

The -noclean option tells this classroom setup to proceed directly with link creation. It does not create initial conditions. Those arrive later, when the FNL script runs WPS and real.exe. Once both steps are complete, the same folder holds the program, runtime data, configuration, and meteorological inputs needed for the forecast.

```
ls -l wrf.exe real.exe LANDUSE.TBL RRTMG_SW_DATA
test -x wrf.exe && echo 'WRF executable is accessible'
```

The actual links in my Erin run folder

This is the complete symbolic-link inventory read from my Erin forecast folder on Amarel. The @ marks a symbolic link, as in ls -F. It is a crowded directory, and that is normal for this setup.

```
/home/lfp33/projs/rumbl_fp/erin_20km
```

```
aerosol.formatted@                                  ETAMPNEW_DATA@
aerosol_lat.formatted@                              ETAMPNEW_DATA.expanded_rain@
aerosol_lon.formatted@                              GENPARM.TBL@
aerosol_plev.formatted@                             grib2map.tbl@
BROADBAND_CLOUD_GODDARD.bin@                        gribmap.txt@
bulkdens.asc_s_0_03_0_9@                            HLC.TBL@
bulkradii.asc_s_0_03_0_9@                           ishmael-gamma-tab.bin@
CAM_ABS_DATA@                                       ishmael-qi-qc.bin@
CAM_AEROPT_DATA@                                    ishmael-qi-qr.bin@
CAMtr_volume_mixing_ratio.A1B@                      kernels.asc_s_0_03_0_9@
CAMtr_volume_mixing_ratio.A2@                       kernels_z.asc@
CAMtr_volume_mixing_ratio@                          LANDUSE.TBL@
CAMtr_volume_mixing_ratio.RCP4.5@                   masses.asc@
CAMtr_volume_mixing_ratio.RCP6@                     MPTABLE.TBL@
CAMtr_volume_mixing_ratio.RCP8.5@                   ozone.formatted@
CAMtr_volume_mixing_ratio.SSP119@                   ozone_lat.formatted@
CAMtr_volume_mixing_ratio.SSP126@                   ozone_plev.formatted@
CAMtr_volume_mixing_ratio.SSP245@                   p3_lookupTable_1.dat-v5.4_2momI@
CAMtr_volume_mixing_ratio.SSP370@                   p3_lookupTable_1.dat-v5.4_3momI@
CAMtr_volume_mixing_ratio.SSP585@                   p3_lookupTable_2.dat-v5.3@
capacity.asc@                                       RRTM_DATA@
CCN_ACTIVATE.BIN@                                   RRTMG_LW_DATA@
CLM_ALB_ICE_DFS_DATA@                               RRTMG_SW_DATA@
CLM_ALB_ICE_DRC_DATA@                               SOILPARM.TBL@
CLM_ASM_ICE_DFS_DATA@                               SOILPARM.TBL_Kishne_2017@
CLM_ASM_ICE_DRC_DATA@                               STOCHPERT.TBL@
CLM_DRDSDT0_DATA@                                   termvels.asc@
CLM_EXT_ICE_DFS_DATA@                               tr49t67@
CLM_EXT_ICE_DRC_DATA@                               tr49t85@
CLM_KAPPA_DATA@                                     tr67t85@
CLM_TAU_DATA@                                       URBPARM_LCZ.TBL@
co2_trans@                                          URBPARM.TBL@
coeff_p.asc@                                        URBPARM_UZE.TBL@
coeff_q.asc@                                        VEGPARM.TBL@
constants.asc@                                      wind-turbine-1.tbl@
create_p3_lookupTable_1.f90-v5.4@                   wrfbdy_d01@
create_p3_lookupTable_2.f90-v5.3@                   wrf.exe@
eclipse_besselian_elements.dat@                     wrfinput_d01@
```

The aerosol, ozone, CO2, greenhouse-gas scenario, land-surface and microphysics files support different model options. Seeing them here does not mean every option is active, or that we are running atmospheric chemistry. Our physics choices determine which resources WRF reads. Leaving the shared runtime links in place is convenient.

Most links in this original folder point into /cache/home/lfp33/repository/WRF/run. The classroom links point into the shared class installation instead, so the exact inventory can differ with the build. The complete original link targets are saved alongside this handout in erin_actual_links.txt.

2. Choosing a domain

For the Erin template I used a 20-km Mercator grid centered near 16.8691°N, 63.3070°W. The choice e_we = 117 and e_sn = 123 gives a projected footprint of 2320 by 2440 km. These are grid dimensions, not distances in kilometers.

Points, intervals, and the reference point

L_x = (e_we – 1) dx = 116 × 20 = 2320 km
L_y = (e_sn – 1) dy = 122 × 20 = 2440 km

The reference latitude and longitude are placed at grid indices ref_x and ref_y. For a centered reference point, ref_x = (e_we + 1)/2 = 59 and ref_y = (e_sn + 1)/2 = 62. Increasing the number of points while keeping dx and dy fixed makes the domain larger. Reducing dx with the same point counts makes it smaller.

```
% In get_wps_domain.m
map_proj = 'mercator';    % alternatively: 'lambert'
dx = 20000;              % meters
dy = 20000;
e_we = 117;
e_sn = 123;
```

The MATLAB script has its own editable settings. We can open it in the Editor, choose the domain, and run the complete script. The following page walks through those choices. It prints the settings and factor statistics in the Command Window and saves a JPEG and PDF in MATLAB's current folder.

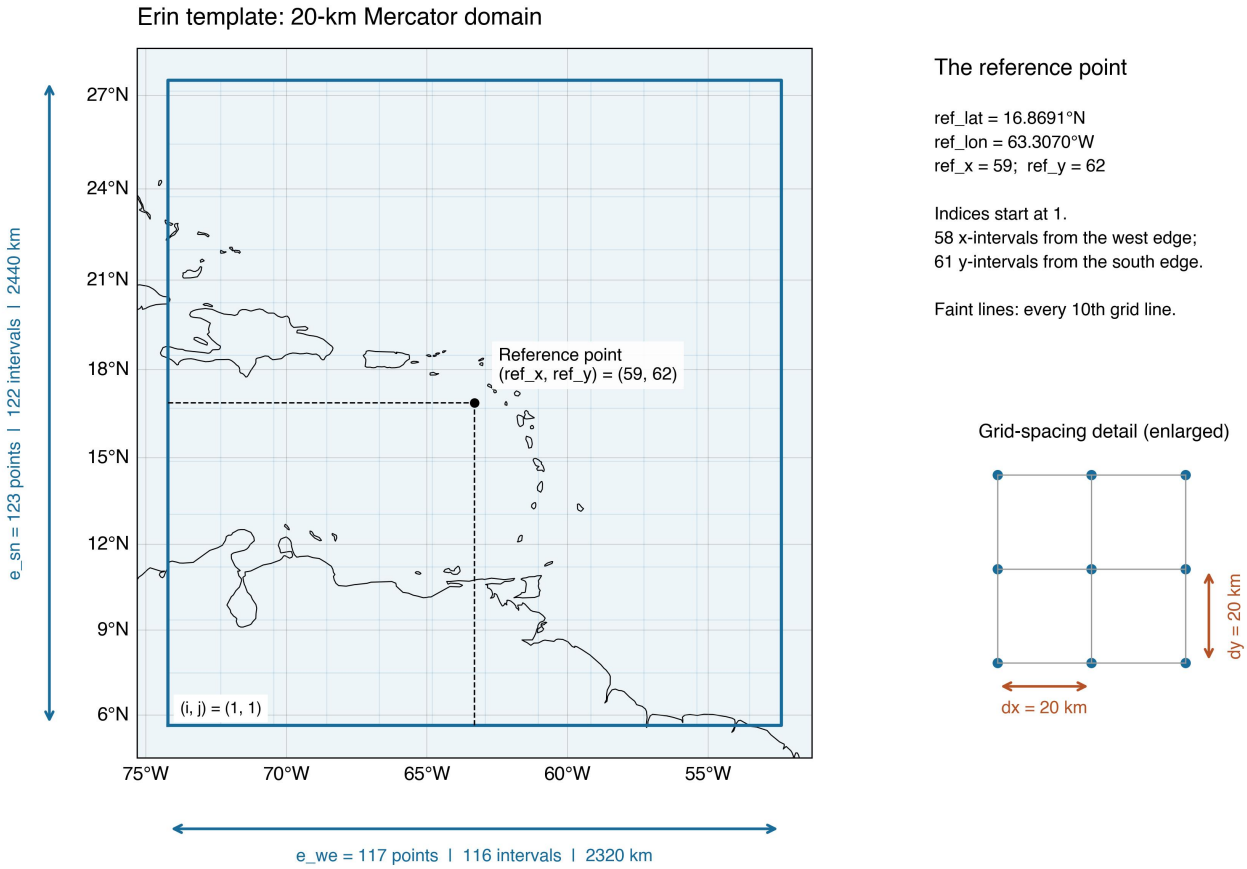
What the map factor means

m = distance on the projection / distance on Earth
For a conformal projection, local ground spacing is approximately dx/m. A nominal 20 km interval where m = 1.08 represents about 18.52 km on Earth.

Mercator is a useful first choice at low latitudes, and Lambert at midlatitudes. That is a starting rule, not an optimization. Inspect the complete domain, including its edges. For Mercator, truelat1 sets the latitude where m = 1; Lambert uses one or two standard parallels. ref_lat sets placement, not the latitude of zero distortion.

Compare projections with care: changing projection changes the footprint even with the same dimensions and center. Confirm that your region of interest remains covered. Green/amber/red thresholds are classroom diagnostics, not universal acceptance limits.

Seeing the grid parameters on a map



The blue arrows span the projected domain. e_we and e_sn count points, so the number of intervals is one fewer. The enlarged detail shows one dx and one dy interval; these small spacings would be difficult to distinguish on the full map.

Connecting geographic and grid coordinates

The black point has two descriptions: ref_lat/ref_lon locate it on Earth, while ref_x/ref_y locate it in the computational grid. For this centered example, it lies 58 intervals east of the western edge and 61 intervals north of the southern edge. The dashed guides help connect the reference point to those edges.

The spacing labels are projected distances. Their ground-distance equivalents vary with map factor. WRF also staggers some variables between grid locations, so an individual output array may have one fewer point than e_we or e_sn in a direction.

Working through the MATLAB map script

With get_wps_domain.m in a local folder, we can select that folder as MATLAB's Current Folder and open the script in the Editor. Its numbered sections separate the choices we make from the projection calculations below.

```
edit get_wps_domain.m
```

1. Projection and spacing

Section 1 lets us choose Mercator or Lambert by leaving one assignment active. In section 2, dx and dy are in meters. For a first comparison, keeping 20000, 117 and 123 preserves the example's nominal spacing and projected size.

```
map_proj = 'mercator';
% map_proj = 'lambert';
dx = 20000; dy = 20000;
e_we = 117; e_sn = 123;
```

2. Position and true-scale latitudes

Section 3 places the reference point. West longitudes are negative. The formulas for ref_x and ref_y keep that point centered when we change the dimensions. Section 4 controls distortion: its true latitudes have a different role from the domain center.

```
ref_lat = 16.869076474579927;
ref_lon = -63.306990521616896;
ref_x = (e_we+1)/2;
ref_y = (e_sn+1)/2;
stand_lon = ref_lon;
mercator_truelat1 = 17.0;
lambert_truelat1 = 8.83;
lambert_truelat2 = 24.21;
```

The Lambert parallels here are examples for the Caribbean. For another region, we can select parallels within its latitude range and compare the resulting factor range.

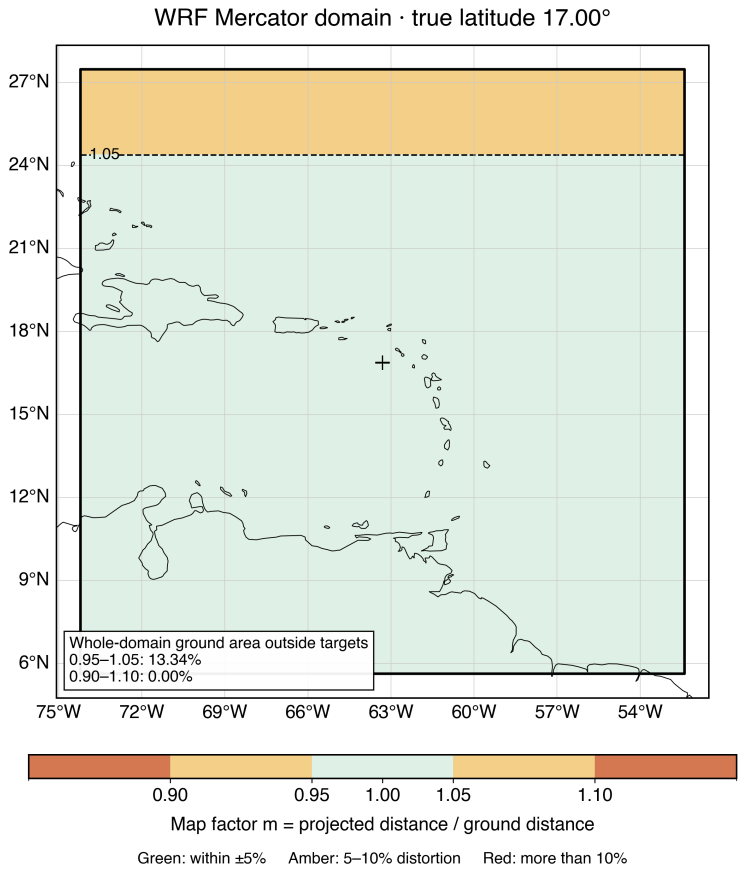
3. Run, inspect, and compare

```
get_wps_domain
```

The Run button does the same thing. The map shows the domain boundary and reference point, with green within ±5%, amber between 5% and 10%, and red beyond 10%. The Command Window reports the minimum and maximum factors, area fractions and ground-spacing range, followed by settings for WPS. The script saves map_scale_factors.jpg at 300 dpi and map_scale_factors.pdf in the current folder.

We can save the first pair under a Mercator name, switch to Lambert, and run again. Saving each result before the next run avoids replacing it. The code uses analytical spherical projections; MATLAB's coastline data are used when available. The Python version remains an alternative for working directly on Amarel.

3. From the map to the domain settings



Example diagnostic for the supplied 20-km Mercator design. The printed summary accompanies the map. The area percentages describe the whole sampled domain, including land; they are not ocean-only statistics.

A short design experiment

We can start with Mercator and note the minimum and maximum factor and the area outside ±5%. A second run with Lambert and suitable standard parallels gives us a useful comparison. Which choice reduces distortion over your area of interest? Does either grid cut off an important upstream weather system or put the feature of interest too close to a lateral boundary?

Once the domain looks suitable, we can open get_wrf_fnl.sh and transfer the final DX, DY, E_WE, E_SN, REF_LAT, REF_LON, REF_X, REF_Y, TRUELAT1, TRUELAT2 and STAND_LON. If we selected Lambert, the literal map_proj in WPS_TEMPLATE also needs to reflect that choice. This keeps the design and preparation settings consistent.

```
nano get_wrf_fnl.sh
# In WPS_TEMPLATE, choose the projection you evaluated:
 map_proj = 'Mercator',
# or:
 map_proj = 'lambert',
```

4. Global weather and our main choices

For the Erin template I requested 14 August 2025 at 00 UTC through 19 August at 00 UTC. The global input consists of NCEP FNL analyses, available here every six hours on a 1° grid. The 20-km WRF grid is finer than the forcing: interpolation alone does not create observed 20-km weather detail.

```
START_DATE = '2025-08-14_00:00:00'
RUN_DAYS = 5
INTERVAL_SECONDS = 21600
CASE_DIR = '.'
DOWNLOAD_DIR = './GRIB'
```

Five days at six-hour intervals require 21 analysis times, including both endpoints. Choose a start at 00, 06, 12 or 18 UTC. The script computes the ending time and writes it into both namelists. WRF uses the sequence to update its lateral boundaries throughout the run.

Dataset information: https://gdex.ucar.edu/datasets/d083002/
Download root used by the script:

```
BASE_URL = 'https://data.gdex.ucar.edu/d083002/grib2'
```

FNL is an analysis product, not a sequence of GFS forecast lead times. The intermediate filename prefix GFS is simply the label used by this WPS workflow; it does not change the provenance of the input. GRIB is a binary format for meteorological fields, with metadata identifying variables, levels, and times.

Read the script in four blocks

Block	What to review
Paths	WPS_DIR and WRF_DIR locate compiled tools. WPS_GEOG locates terrain and land datasets. CASE_DIR keeps products beside your script.
Stage switches	GET_FNL, RUN_GEOGRID, RUN_UNGRIB, RUN_METGRID, RUN_REAL: 1 runs the stage; 0 attempts to reuse checked existing products. Keep all at 1 initially.
Domain and clock	Dates, dimensions, spacing, center, projection parameters, and TIME_STEP.
Embedded namelists	WPS_TEMPLATE and WRF_TEMPLATE hold the settings written to namelist.wps and namelist.input.

The generated namelists are useful to inspect, but the script writes them again each time it runs. Our lasting changes therefore belong in the shared values near the top or in the relevant embedded template.

5. From a prepared grid to model physics

namelist.wps controls the preparation grid and input sequence. namelist.input is introduced at real.exe: it controls the vertical model grid and initialization, and later controls time integration and physics in wrf.exe. Both describe the same application.

Setting	Meaning in this exercise
e_vert = 40	40 vertical interfaces and 39 mass layers; these are not the FNL pressure levels.
p_top_requested = 10000	Top pressure of 10000 Pa (100 hPa).
num_metgrid_levels = 34	Expected FNL-derived metgrid level count; check the actual met_em header if changing the data source.
time_step = 10	A conservative 10-s example; stability also depends on winds, terrain, and map factors.
sst_update = 0	No time-varying prescribed SST update file in this template.
sst_skin = 1	Retained standalone skin-temperature treatment; distinct from changing the SST input dataset.

Why convection and microphysics both appear

At 20 km I do not expect an individual convective updraft to be resolved. I therefore keep Kain–Fritsch convection alongside WSM6 microphysics. WSM6 represents grid-scale water and ice processes; the cumulus scheme represents effects of unresolved convection. Their roles overlap physically, but they are not interchangeable switches.

```
&physics
 mp_physics      = 6,       ! WSM6
 cu_physics      = 1,       ! Kain-Fritsch
 cu_rad_feedback = .true.,  ! convective cloud-radiation coupling
 icloud          = 1,       ! cloud effects in radiation
 ra_sw_physics   = 4,       ! RRTMG shortwave
 ra_lw_physics   = 4,       ! RRTMG longwave
 bl_pbl_physics  = 5,       ! MYNN boundary layer
 sf_sfclay_physics = 5,     ! MYNN surface layer
/
```

For this class template I explicitly enable cu_rad_feedback. This is a change from the original Erin feedback-off baseline. With the supported KF/RRTMG combination it permits parameterized convective clouds to affect radiation. icloud = 1 keeps cloud radiative effects enabled; it does not turn on convection or guarantee realistic solar flux.

The displayed block highlights selected settings, not the entire namelist. MYNN option 5 alone should not be read as proof that every optional EDMF control is enabled. Inspect explicit options and namelist.output in any forecast. A 3-km experiment requires a fresh assessment of convection treatment, time step, cost, and domain coverage.

6. A preview and the first submission

Before consuming an allocation, inspect the plan and generated settings. These commands do not download the forcing or run WPS:

```
bash get_wrf_fnl.sh --plan
bash get_wrf_fnl.sh --render-only
less namelist.wps
less namelist.input
```

Before submitting, we can open submit.sh and replace the email placeholder, including its angle brackets, with our own address. The copied file intentionally contains:

```
#SBATCH --mail-user=<enter your email>
#SBATCH --mail-type=BEGIN,END,FAIL
```

The address goes on that line without spaces. The partition/account settings depend on your Amarel account; the template uses your default partition. It requests one task on one shared node, 32 GB, and 12 hours, without an exclusive-node request.

```
nano submit.sh
sbatch submit.sh
squeue -u "$USER"
tail -f log.wps
```

The submission loads nco and roms, runs get_wrf_fnl.sh, and then runs verify_wrf_inputs.py. The master script launches its own one-task srun stages. WPS preparation and downloads can take substantial time; queue waiting is separate. Stop tail with Ctrl-C; this does not cancel the job.

Expected success markers, not a shortened replacement log

```
Successful completion of geogrid
Successful completion of ungrib
Successful completion of metgrid
SUCCESS COMPLETE REAL_EM INIT
```

The full Slurm standard-output file is log.wps.JOBID, with log.wps pointing to it. Native geogrid.log, ungrib.log, metgrid.log and real.exe rsl files remain available; logs/ also preserves per-attempt records and a summary. A file’s presence alone is not proof of success: confirm the marker, exit status, dates, and fields.

If plotting fails after real.exe succeeds, inspect err.wps.JOBID and rerun only the verification command after fixing that error. Do not redownload everything to fix a plotting dependency. Warnings require interpretation; a fatal error or missing success marker requires investigation.

7. What appears in your application folder

```
exercise/
  build_wrf.sh       get_wrf_fnl.sh       submit.sh
  get_wps_domain.m   verify_wrf_inputs.py
  wrf.exe -> shared compiled WRF
  real.exe -> shared compiled WRF
  geogrid.exe  ungrib.exe  metgrid.exe    [links]
  namelist.wps      namelist.input
  GRIB/             geo_em.d01.nc
  GFS:...           met_em.d01.*.nc
  wrfinput_d01      wrfbdy_d01
  wrfinput_d01.nc -> wrfinput_d01
  wrfbdy_d01.nc   -> wrfbdy_d01
  log.wps           logs/
  verification/
```

The .nc names are visible NetCDF aliases, not extra copies. The classroom namelist reads the .nc names explicitly. Each alias still points to its conventional-name target, so both stay together in this folder. If transferring the .nc files to a separate run folder, copying with cp -L copies the data behind the links. The files were already NetCDF before adding the extension.

Verification figures at 300 dpi

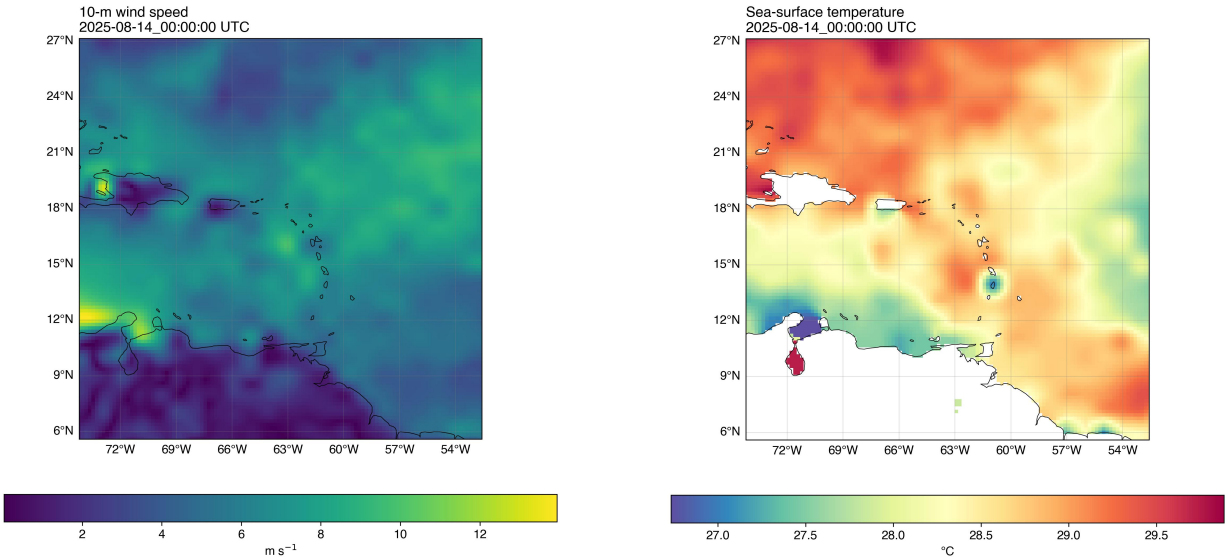
```
/home/lfp33/miniconda3/envs/mercator2/bin/python verify_wrf_inputs.py
```

JPEG / diagnostic	What to ask
wind10.jpg	Is the 10-m wind-speed pattern plausible for this initial time?
sst.jpg	Is SST plausible in °C, masked over land, and spatially coherent?
air_temperature.jpg	Does 2-m air temperature reflect land, ocean, and elevation?
surface_pressure.jpg	Is PSFC in hPa plausible? This is surface pressure, not reduced sea-level pressure.
humidity.jpg; terrain.jpg	Are water-vapor mixing ratio and terrain physically sensible?
map_factor.jpg	Do actual initialized map factors agree with your design estimate?

The verification folder also contains input_variables.csv and verification_summary.json: variable names, units, dimensions, ranges, and boundary timestamps. Current speed is plotted only if UOCE and VOCE exist; their presence alone does not establish realistic ocean currents in a standalone FNL case.

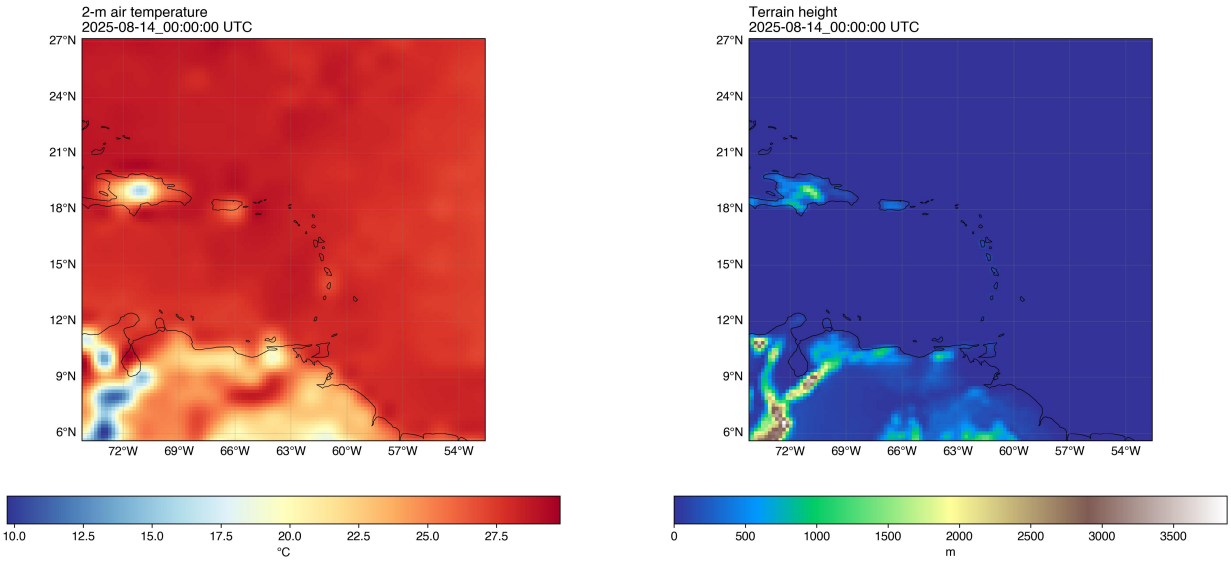
8. Making sense of the initial fields

Initialization should preserve a sensible large-scale atmosphere. It need not contain a mature, sharply resolved hurricane eyewall at the beginning of the simulation. Features may evolve during adjustment; that is not a reason to overlook a wrong date, misplaced domain, or erroneous temperature conversion.



Actual Erin initial conditions, 14 August 2025 at 00 UTC, drawn by the supplied verification script. These are initial fields, not a forecast or evidence of the new radiation-feedback setting's effect.

Two more views of the initialized surface



Air temperature at 2 m and terrain height give us another useful comparison: how do the land, ocean, and mountain patterns relate? These are actual Erin initialization fields from the verification script.

Discussing your application

Bringing your application together

A domain map, the verification figures, and a short table of the projection, center, dx/dy, dimensions, dates, map-factor range, and physics choices give us a useful record of the application. Together, these help us discuss whether the domain suits our question and what we would investigate before trusting a forecast.

For boundaries, check that the first time matches initialization and that the interval endpoints cover the requested ending time. The script records thisbdytime and nextbdytime when present; the last boundary-record timestamp alone need not equal the final endpoint.

Where to learn more

FNL dataset and provenance
https://gdex.ucar.edu/datasets/d083002/

WPS: domains, projection parameters, geogrid, ungrib, metgrid
WRF Users' Guide: The WRF Preprocessing System

Physics: supported schemes and interactions
WRF Users' Guide: Physics

Namelist reference and running the model
WRF Users' Guide: Running WRF

Shared classroom executables: WRF v4.5.1-17-gee6e88c9 and WPS v4.7.0. Consult documentation appropriate to the executable version when adding options. Successful preprocessing establishes usable inputs; observational evaluation is still needed to assess forecast accuracy.

The forecast clock and time step

Once preparation is finished, namelist.input tells WRF when to start and how long to integrate. For our Erin example, the clock starts at 00 UTC on 14 August 2025 and advances for five simulated days.

```
&time_control
 run_days    = 5,
 run_hours   = 0,
 run_minutes = 0,
 run_seconds = 0,
 start_year  = 2025,
 start_month = 8,
 start_day   = 14,
 start_hour  = 0,
 end_year    = 2025,
 end_month   = 8,
 end_day     = 19,
 end_hour    = 0,
/
```

run_days is the number of simulated days, not the job's wall-clock allowance. For a 36-hour forecast, for example, run_days = 1 and run_hours = 12. The start must match wrfinput_d01.nc, and wrfbdy_d01.nc must cover the full duration. WRF gives a specified run duration precedence over the end-date fields; keeping them consistent makes the intended clock clear. real.exe uses the start and end dates when preparing the inputs.

A starting rule for time_step

Time-step rule of thumb

time_step [seconds] ≈ 6 × grid spacing [kilometers]

20 km → about 120 s | 3 km → about 18 s

This is a starting estimate for typical cases. Strong winds, steep terrain, vertical motion, and map-scale distortion can require a smaller step. For unequal dx and dy, the smaller spacing is the conservative reference.

Our Erin template keeps the conservative time_step = 10 s used in the example. The rule does not establish that 120 s will be stable for this hurricane. time_step belongs in &domains; it controls the integration, independently of how often we save output.

How often will we save the model state?

Before the forecast, we can choose output frequency and filenames in the &time_control section of namelist.input. These settings control simulated time intervals.

```
history_interval   = 60,       ! minutes between saved states
frames_per_outfile = 24,       ! saved states in each file
history_outname    = "my_wrf_d<domain>_<date>.nc",
restart            = .false.,  ! this run starts from initial conditions
restart_interval_d = 1,        ! save a restart every simulated day
rst_outname        = "wrfrst_d<domain>_<date>",
```

Here, history output is hourly, with 24 saved states per file. frames_per_outfile = 1 instead gives one file per saved time. A 30-minute history interval gives twice as many snapshots as an hourly interval over the same forecast. The filename records the first saved time in each file.

Using WRF's standard output names

A leading exclamation mark comments out a namelist line. If we comment out history_outname, WRF uses its standard wrfout naming pattern:

```
! history_outname = "my_wrf_d<domain>_<date>.nc",

! Example standard filename:
! wrfout_d01_2025-08-14_00:00:00
```

The <domain> and <date> tokens are filled in by WRF. The supplied template already sets history_outname to the standard pattern, so commenting out that existing line leaves the familiar wrfout* names. io_form_history = 2 writes NetCDF even if the filename has no .nc suffix.

Restart files

Restart files contain the state needed to continue a simulation. restart = .false. means this is a fresh run; it can still write restart files. To save them every six hours, we can replace restart_interval_d = 1 with restart_interval = 360, whose units are minutes. Using one interval form avoids ambiguity. A later continuation uses restart = .true. with the appropriate restart files and clock.

These forecast-only settings can be edited in namelist.input after preparation. If we rerun get_wrf_fnl.sh, it regenerates the namelist; changes we want to retain through preprocessing belong in its WRF_TEMPLATE block.

Timing and output options are documented in the WRF Users' Guide namelist reference.

Ready for WRF: the practical checklist

For this single-domain, fresh-start exercise, the forecast needs the following files in our run folder. The WPS work is complete: wrf.exe does not need geo_em, met_em, GRIB files, or namelist.wps.

File or link	Quick check
[ ] wrfinput_d01.nc	Initial conditions exist, are readable, and have the intended start time.
[ ] wrfbdy_d01.nc	Boundary conditions exist, are readable, and cover the requested number of days.
[ ] namelist.input	Start date and run_days agree with the inputs; history/restart intervals and filenames are what we want.
[ ] wrf.exe + runtime links	wrf.exe is executable, and the tables/data created by build_wrf.sh -noclean are accessible.
[ ] submit.sh	The forecast job script loads our modules and runs srun --ntasks=32 ./wrf.exe. Our email and job resources are set.

The classroom namelist names the .nc input files explicitly:

```
 input_inname = "wrfinput_d01.nc",
 bdy_inname   = "wrfbdy_d01.nc",
```

To give the forecast job the simple name submit.sh, we can save the earlier preparation job as submit_prep.sh and copy the forecast template:

```
mv submit.sh submit_prep.sh
cp /projects/f_omg_1/lfp33/nmoa_2026/wrf_exercise/submit_wrf.sh submit.sh
nano submit.sh
ls -l wrfinput_d01.nc wrfbdy_d01.nc namelist.input submit.sh wrf.exe
sbatch submit.sh
```

The .nc aliases need their targets beside them. The forecast template checks that both input files are readable and nonempty. It uses 32 tasks on one shared node. Once submitted, we can follow rsl.out.0000 and rsl.error.0000; a successful finish reports SUCCESS COMPLETE WRF.